

SwarmForge - Decentralized 3D Printing Network

Overview

SwarmForge is a sophisticated decentralized 3D printing system that enables distributed manufacturing by splitting large 3D models into parts and distributing them across a network of certified printers for parallel production and faster delivery.

Architecture

Microservices

1. **API Gateway** (Go) - Entry point, authentication, rate limiting
2. **Job Orchestrator** (Go) - Job distribution, printer matching, workflow coordination
3. **Printer Service** (Go) - Printer registration, certification, capability tracking
4. **QA Service** (Node.js/TypeScript) - Quality verification, photo analysis, dimensional checks
5. **Model Processor** (Go) - 3D model slicing, part optimization, STL processing
6. **WebSocket Server** (Node.js) - Real-time updates, printer status, job progress
7. **Frontend** (Next.js/TypeScript) - User interface, dashboard, admin panel

Tech Stack

- **Backend:** Go (performance-critical services), Node.js/TypeScript (real-time, QA)
- **Frontend:** Next.js, React, TypeScript, TailwindCSS
- **Database:** PostgreSQL (relational data), Redis (caching, queues)
- **Message Queue:** RabbitMQ or NATS
- **Storage:** MinIO (S3-compatible) for 3D models and QA photos
- **Container Orchestration:** Docker, Kubernetes

Key Features

1. Printer Certification System

- **Tier Classification**
 - Platinum: Industrial (Bambu X1C, Prusa XL, etc.)
 - Gold: Prosumer (Bambu P1S, Prusa MK4)
 - Silver: Consumer (Ender 3 V3, etc.)
- **Calibration Verification**
 - Automated test prints (dimensional accuracy tests)
 - Photo verification of calibration cubes
 - Tolerance measurements ($\pm 0.1\text{mm}$, $\pm 0.05\text{mm}$, etc.)
 - Material compatibility verification
- **Reputation System**

- Success rate tracking
- QA pass rates
- Delivery time compliance
- User ratings

2. Job Distribution Algorithm

1. Analyze 3D model complexity
2. Partition into optimal parts (size, orientation, support)
3. Match parts to printer capabilities:
 - Print volume
 - Material availability
 - Quality tier requirements
 - Geographic proximity (shipping optimization)
4. Assign parts with redundancy (1.2x for critical parts)

3. Quality Assurance Workflow

Pre-Print QA

- Model validation (manifold, printability)
- Slicer settings review
- Material compatibility check

Post-Print QA

1. Photo Verification

- Multi-angle photos required
- AI-powered defect detection
- Surface quality assessment

2. Dimensional Verification

- Caliper measurements for critical dimensions
- Comparison against CAD model
- Tolerance validation

3. Functional Testing (when applicable)

- Fit testing with mating parts
- Stress testing for structural parts

4. Final Assembly Verification

- All parts fit together
- No warping or dimensional drift
- Acceptance by requester

QA Decision Tree

```
Photo Upload → AI Analysis → Pass/Flag
↓
Manual Review (if flagged)
↓
Dimensional Check
↓
Accept/Reject/Request Reprint
↓
Payment/Reputation Update
```

4. User Roles

- **Requesters:** Submit models, receive assembled products
- **Printers:** Register machines, accept jobs, submit QA
- **QA Reviewers:** Manual verification, dispute resolution
- **Admins:** System oversight, certification approval

Getting Started

Prerequisites

- Go 1.21+
- Node.js 20+
- Docker & Docker Compose
- PostgreSQL 15+
- Redis 7+

Installation

```
# Clone repository
git clone <repo-url>
cd Proj-Swarmforge

# Install dependencies
make install

# Setup environment
cp .env.example .env

# Start services
docker-compose up -d

# Run migrations
make migrate
```

```
# Start development
make dev
```

API Endpoints

Jobs

- `POST /api/v1/jobs` - Submit new print job
- `GET /api/v1/jobs/:id` - Get job status
- `GET /api/v1/jobs/:id/parts` - List job parts

Printers

- `POST /api/v1/printers` - Register printer
- `POST /api/v1/printers/:id/certify` - Submit certification test
- `GET /api/v1/printers/available` - Find available printers

QA

- `POST /api/v1/qa/submit` - Submit QA photos/measurements
- `POST /api/v1/qa/review/:id` - Review QA submission
- `GET /api/v1/qa/pending` - Get pending QA items

Database Schema

See [docs/schema.md](#) for detailed database design.

Contributing

See [CONTRIBUTING.md](#)

License

MIT